

Estructuras de Datos y Algoritmos

Práctica I – Curso 2026/27

Asignación de Servicios (I)

1. Introducción

En la práctica de este curso vamos a examinar dos problemas relacionados con una pequeña parte de una aplicación real (asignación óptima de recursos sanitarios) y veremos que el escoger las estructuras de datos y los algoritmos adecuados puede tener un impacto muy importante en la eficiencia de la aplicación.

Dispondremos de los datos de la **red de carreteras**, **núcleos** de población, y posibles puntos donde se pueden situar instalaciones sanitarias (**puntos de servicio**) referentes a una determinada área geográfica (dispondremos de datos de áreas geográficas cada vez mayores hasta cubrir toda Castilla y León). La red de carreteras se representará como un grafo donde las aristas representan tramos de carreteras y los vértices cruces de carretera (puntos donde se unen dos o más tramos) o puntos finales (comienza o se termina la carretera).

Los problemas que vamos a resolver son muy sencillos: El **primero** es el de **asignar cada punto de servicio al cruce más cercano** desde el punto de vista geográfico (es decir, determinar el punto desde el cual se accede desde ese punto a la red de carreteras), y el **segundo** el de **asignar cada núcleo de población al punto de servicio más cercano**, pero ahora en vez de usar como criterio la cercanía geográfica (a "vuelo de pájaro") usaremos la cercanía usando **la red de carreteras**.

Estos mini-problemas forman parte de un problema de optimización mucho más complejo (y del que no nos vamos a preocupar), y en concreto en este problema general, fijada una zona geográfica, se irán generando distintos conjuntos de posibles puntos de servicio para ser evaluados, y como parte de esa evaluación se encuentran los mini-problemas que vamos a resolver.

En esta primera práctica se indicará en detalle las estructuras de datos y algoritmos a utilizar, y representan las elecciones "obvias" que se tomarían si no se analizan a fondo los problemas planteados. El objetivo es obtener una estimación de la eficiencia de esta solución en función del tamaño de los datos, es decir ver como escalará el tiempo a medida que vamos trabajando con zonas geográficas más amplias que requieren más datos para su representación.

2. Descripción detallada

2.1. Objetivos de la práctica

Los objetivos de la práctica son el **crear una aplicación Java** que lea los datos asociados a una zona geográfica y resuelva los problemas planteados (asignación de cruce a servicio y asignación de servicio a núcleo), y el uso de esta aplicación para **tomar una serie de medidas del tiempo** que tarda la aplicación. Con estas medidas intentaremos estimar la **complejidad temporal en función del tamaño** de los datos de nuestra solución, presentando un informe con las conclusiones.

La ejecución de la aplicación se dividirá en cuatro etapas principales:

1. **Lectura de los datos** del problema del directorio que le ha indicado el usuario.
2. Cálculo de las **listas de adyacencia** (necesarias para la resolución del segundo problema)
3. Resolución del **primer problema**
4. Resolución del **segundo problema**

La ejecución se repetirá varias veces de forma que podamos obtener una estimación del tiempo promedio de cada etapa.

Por supuesto antes de obtener las medidas se deberá comprobar que la aplicación funciona correctamente (se proporcionarán las soluciones de cada escenario).

2.2. Estructura de la aplicación

Para el desarrollo de la aplicación se han depositado en el Campus Virtual una serie de ficheros Java correspondientes a las siguientes clases:

- Cuatro clases que representan a los elementos fundamentales del problema: **Cruces** y **tramos** para la red de carreteras, **núcleos** de población y puntos de posible apertura de **servicio**.
- Una clase abstracta, **SolucionAbs**, que declara las cuatro listas (de cruces, tramos, núcleos y servicios) que representan los datos del problema, los cuatro métodos que realizan las etapas en las que se va a dividir el problema (lectura de ficheros, cálculo de las listas de adyacencia, primer problema y segundo problema), y tres métodos adicionales (algoritmo de Dijkstra, distancia geográfica entre puntos y escritura de resultados). Los métodos de lectura de fichero, distancia geográfica y escritura de resultados están implementados y el resto son abstractos.

Debéis utilizar **obligatoriamente** estos ficheros y **no podéis modificar** su código de ninguna forma.

Además de las clases que se os proporcionan, vosotros debéis crear al menos las siguientes clases:

- **Solucion**: Esta clase debe heredar de **SolucionAbs** y vuestro objetivo es implementar los métodos referentes a las etapas de cálculo de las listas de adyacencia, resolución del primer problema, resolución del segundo problema y la implementación del algoritmo de Dijkstra (necesario para el segundo problema).
- **ListaOrd**: Esta clase representa un conjunto de cruces (estructura necesaria para el algoritmo de Dijkstra) y se implementará mediante un array parcialmente lleno ordenado de acuerdo a un criterio particular.
- **Evaluador**: Esta es la clase principal de la aplicación, consiste en un único método **main** que crea el objeto de clase **Solucion** y ejecuta las etapas del problema llamando a sus métodos y midiendo su tiempo de ejecución.

2.3. Conjunto de datos

Para cada zona geográfica dispondremos de cuatro listas de datos, que se han simplificado para que solo contengan la información estrictamente necesaria:

- La lista de **cruces** de la red de carreteras (fichero `cruces.txt`). Cada cruce está representado por la clase **Cruce** y almacena únicamente su **posición geográfica** (latitud y longitud).
- La lista de **tramos** de la red de carreteras (fichero `tramos.txt`). Cada tramo une dos cruces, el cruce **origen** y el cruce **destino**, representados por su **índice** en la lista de cruces. Cada tramo es direccional (**de sentido único**), para carreteras de doble sentido existirán dos tramos en la lista (con los cruces origen y destino invertidos). Además del cruce origen y destino la clase **Tramo** almacena la información del **tiempo** que se tarda en recorrerlo.
- La lista de **núcleos** de población (fichero `nucleos.txt`). Cada núcleo de población está localizado en un **cruce** de la red de carreteras, y los datos almacenados en la clase **Nucleo** son el **índice** de ese cruce y el **nombre** de la localidad.
- La lista de **posibles puntos de servicio** (fichero `servicios.txt`), representados por la clase **Servicio**, que almacena su **posición geográfica** (latitud y longitud) y el **nombre** del servicio.

Se proporciona el método **LeeFicheros** en la clase **SolucionAbs** para leer los ficheros asociados a una zona geográfica y generar las cuatro listas que almacenan los datos del problema (cruces, tramos, núcleos y servicios).

2.4. Listas de Adyacencia

Las listas de adyacencia son una estructura de datos que se utilizan para representar un grafo almacenando, para cada nodo, la lista no ordenada de sus nodos adyacentes (para los que existe una arista que conecta directamente ese nodo con el adyacente). Para nuestro problema la implementaremos como un array indexado por cruces y cuyos elementos son listas (`ArrayList`) de objetos `Tramo`: La lista de tramos que tienen como punto de origen ese cruce. Usaremos esta estructura de datos en el algoritmo de Dijkstra (resolución del segundo problema), pero hemos separado su cálculo en una etapa aparte porque solo necesita calcularse una vez, ya que no depende de la parte variable del problema general (los puntos de servicio).

En la Fig. 1 se muestra un conjunto de datos de test, los cruces se representan mediante círculos con el índice del cruce en el interior, los tramos como líneas azules que conectan cruces (en rojo el tiempo que se tarda en recorrerlo), los núcleos de población se encuentran en los cruces cuyo fondo es gris (se indica su nombre) y por último los puntos de servicio se muestran como hexágonos (no están situados exactamente sobre un cruce, en el primer problema tenemos que encontrar su cruce más cercano geográficamente). **Nota:** Los tramos de doble sentido, aunque se muestran como una única línea en la figura, son en realidad **dos tramos**, uno en cada dirección, y es posible (a diferencia de lo que pasa en la figura) que el tiempo de recorrido sea distinto en cada tramo.

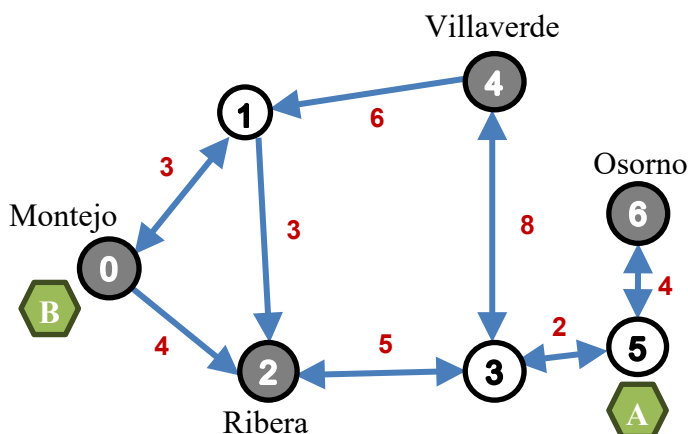


Fig. 1 Conjunto de datos de test

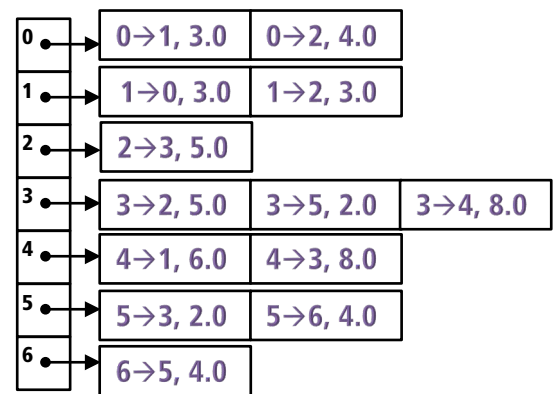


Fig. 2 Listas de adyacencia del conjunto de datos test

En la Fig. 2 se muestra la lista de adyacencia (array de listas de tramos) que obtendríamos al procesar los datos del conjunto de test. A continuación se indica el algoritmo para implementar esta etapa (método **CreaAdyacencia** de vuestra clase **Solucion**):

Cálculo de las listas de adyacencia	
Creamos un array donde para cada cruce almacenamos la lista de tramos que tienen ese cruce como origen.	
Variable	Descripción
n	Número de cruces
<pre> function CreaAdyacencia(Tramos, n): <i>ady</i> ← array[0..<i>n</i>-1] de listas de tramos vacías for each tramo <i>t</i> de Tramos: <i>u</i> ← índice de cruce origen de <i>t</i> añadir a lista <i>ady</i>[<i>u</i>] el tramo <i>t</i> return <i>ady</i> </pre>	

Tab. 1: Algoritmo de la segunda etapa

2.5. Primer Problema: Asignación servicio-cruce

En la aplicación de optimización real la localización de los puntos de servicio es “dinámica” y en cierto modo independiente de la localización de los núcleos de población y de la red de carreteras. Por eso es necesario, antes de resolver el problema de asignación núcleos-servicios, encontrar el punto (cruce) de la red de carreteras desde donde se accede al servicio.

Se muestra a continuación el algoritmo que vamos a usar en la primera práctica, que consiste simplemente en asignar a cada servicio su cruce **más cercano geográficamente**. Para realizar la comparación os hemos proporcionado la función **DistQ2** en la clase **SolucionAbs**, que devuelve una estimación de la distancia entre dos puntos geográficos (indicados por longitud y latitud).

Algoritmo del Primer Problema	
Para cada punto de servicio recorremos los cruces calculando la distancia geográfica de cada cruce al servicio para asignar al servicio el cruce más cercano.	
Variable	Descripción
n	Número de cruces
<pre> function Problema1(Cruces, Servicios): for each servicio s de Servicios: d_min ← ∞ u_min ← -1 for each u ∈ [0..n-1]: dist ← DistQ2(posición de s, posición de Cruces[u]) if dist < d_min: d_min ← dist u_min ← u asignar a s el cruce u </pre>	

Tab. 2 Algoritmo de asignación cruce a servicio

La asignación del cruce al servicio se realiza mediante el setter **setCruce** de la clase **Servicio**, que actualiza su atributo **indCruce** (al que se puede acceder con el getter **getCruce**).

2.6. Algoritmo de Dijkstra

Para la resolución del segundo problema necesitamos, para cada núcleo, calcular los caminos óptimos (tiempo mínimo) a todos los puntos de servicio, encontrando el servicio más cercano por la red de carreteras. Existen muchos algoritmos para calcular el camino óptimo, pero Dijkstra parece una opción razonable ya que calcula, dado un nodo origen, el camino óptimo al resto de nodos. Para nuestro caso calcularíamos el camino desde un cruce (el del núcleo) al resto de cruces y recorreríamos los cruces de los servicios para determinar el de menor distancia. Otros algoritmos como la búsqueda A* (que calcula el camino óptimo desde un nodo origen a un nodo destino) o Floyd-Warshall o Johnson (caminos óptimos de todos contra todos) no se adaptan tan bien a nuestro problema (desde el punto de vista de la eficiencia) y soluciones como *contraction hierarchies* son mucho más complejas de implementar.

Como en nuestro problema no necesitamos reconstruir el camino óptimo sino únicamente conocer su distancia, el algoritmo de Dijkstra lo implementaremos de acuerdo al siguiente pseudocódigo en el método **Dijkstra** de nuestra clase **Solucion**:

Algoritmo de Dijkstra	
Recibe la descripción de la red de carreteras (listas de adyacencia) y un punto de origen (cruce) y devuelve los tiempos de los caminos óptimos desde el cruce de origen al resto de los cruces	
Variable	Descripción
n	Número de cruces
Ady	$array[0..n-1]$ de listas de tramos
$Ady[u]$	Lista de tramos que tienen por origen el cruce u
fnt	El cruce de origen
$dist$	$Array[0..n-1]$ de tiempos del camino mínimo
$dist[u]$	Tiempo del camino mínimo $fnt \rightarrow u$
Q	Conjunto de cruces que quedan por visitar
<pre> function Dijkstra(Ady, fnt): crear $dist$ y Q for each cruce u: $dist[u] \leftarrow \infty$ añadir u a Q $dist[fnt] \leftarrow 0$ actualizar Q por cambio en $dist[fnt]$ while no vacío Q: $u \leftarrow$ cruce de Q con menor $dist[u]$ quitar u de Q for each tramo t de $ady[u]$: $v \leftarrow$ cruce destino de t $alt \leftarrow dist[u] + longitud\ (tiempo)\ de\ t$ if $alt < dist[v]$: $dist[v] \leftarrow alt$ actualizar Q por cambio en $dist[fnt]$ return $dist$ </pre>	

Para representar el conjunto de cruces Q habitualmente se utiliza una cola de prioridad, pero en esta primera práctica vamos a utilizar una estructura de datos más sencilla que detallamos en la siguiente sección.

2.7. Representación del Conjunto de Cruces en Dijkstra

El bucle principal del algoritmo de Dijkstra pasa por todos los nodos (cruces) una única vez, y utiliza una estructura de datos (el conjunto de cruces Q) para controlar el recorrido. Las operaciones necesarias sobre esta estructura se han marcado en rojo en el algoritmo, y las más importantes son el encontrar el cruce cuya distancia actual al origen (array $dist$) sea mínima, y el actualizar la estructura cuando modifiquemos algún valor de $dist$.

La representación que vamos a elegir es un **array de índices de cruces parcialmente lleno y ordenado** de acuerdo al array $dist$ (en orden inverso, de mayor a menor), y lo implementaremos en la clase **ListOrd**.

La estructura constará de un array de enteros (índices de cruces) **v**, de un entero **n**, y de una referencia al array **dist** (o bien de un objeto de clase **Comparator** definido en función de ese array). El array **v** tendrá una determinada capacidad pero en un momento dado solo serán válidos los valores **v[0..n-1]**. La condición de que este ordenado de mayor a menor significa que se debe cumplir que **dist[v[i-1]] ≥ dist[v[i]]**.

La forma de implementar las operaciones necesarias en **ListaOrd** es la siguiente:

- Comprobación de **conjunto vacío**: Simplemente se comprueba si **n = 0**.
- **Añadir** el cruce de índice **c** al conjunto: Se usa búsqueda binaria sobre **v[0..n-1]** para encontrar el índice **i** donde se cumpla que **dist[v[i-1]] ≥ dist[c] > dist[v[i]]**, se desplazan una posición a la derecha los elementos **v[i..n-1]** y se asigna **v[i] = c**. Por supuesto, primero se comprueba si hay sitio en el array y en caso contrario se amplía duplicando su capacidad.
- **Devolver y borrar el cruce mínimo**: Por la forma en que hemos diseñado la estructura estas operaciones son triviales: El cruce mínimo es el último (**v[n-1]**) y se elimina decrementando **n**.
- **Actualizar** la estructura cuando existe un cambio en **dist**: En este caso podemos hacer uso de que en nuestro problema se realiza un único cambio en una posición de **dist** y ese cambio siempre **disminuye** su valor. Como no sabemos la posición de **v** donde se ha producido el cambio debemos recorrer el vector comprobando valores contiguos hasta que encontremos que no se cumpla la condición de ordenación. En el siguiente diagrama se muestra la situación cuando el decremento se ha producido en el cruce cuyo índice está en **v[i]**:



En el diagrama **x** representa **dist[v[i]]**, y vemos que su disminución ha provocado que la zona **v[i+1..k]** sean elementos mayores que **x** cuando deberían ser menores al estar a su derecha (recordemos que el orden es de mayor a menor). La forma de resolverlo es encontrar **k+1**, el índice del primer elemento que vuelve a ser menor que **x**, desplazar el subvector **v[i+1..k]** una posición a la izquierda y por último insertar **v[i]** en **v[k]**.

2.8. Segundo Problema: Asignación núcleo-servicio

Como ya se ha comentado, el algoritmo consiste en un bucle principal en el que se recorren los núcleos de población, se usa el algoritmo de Dijkstra con el cruce del núcleo como origen para obtener el vector que indica el tiempo del camino óptimo desde ese cruce a todos los demás, y en un bucle interno se recorren los servicios para encontrar aquel cuyo cruce sea más cercano (el tiempo sea el menor).

La clase **Nucleo** dispone de dos atributos, **indServicio** y **tpoServicio**, para almacenar esa información, que se asigna mediante el setter **setServicio**.

Algoritmo del Segundo Problema	
Para cada núcleo de población llamamos a Dijkstra para que calcule el camino óptimo desde ese núcleo a todos los cruces, y recorremos los servicios escogiendo aquel cuyo cruce sea el más cercano (menor tiempo) al cruce del núcleo.	
Variable	Descripción
<i>m</i>	Número de servicios

```

function Problema2(Ady, Nucleos, Servicios):
  for each nucleo  $p$  de Nucleos:
     $u \leftarrow$  cruce asignado al núcleo  $p$ 
     $dist \leftarrow$  Dijkstra(Ady,  $u$ )
     $tpo\_min \leftarrow \infty$ 
    for each  $i \in [0..m-1]$ :
       $v \leftarrow$  cruce asignado al servicio Servicios[ $i$ ]
      if  $dist[v] < tpo\_min$ :
         $tpo\_min \leftarrow dist[v]$ 
         $ind\_min \leftarrow i$ 
    asignar a  $p$  el servicio  $ind\_min$  con tiempo  $tpo\_min$ 

```

2.9. Tamaños de las entradas

Al tener cuatro fuentes de datos la eficiencia será una función que dependerá de 4 variables: número de cruces (N_c), número de tramos (N_t), número de núcleos (N_n) y número de servicios (N_s). Afortunadamente en este problema todas estas variables crecen aproximadamente al mismo ritmo, y los factores $K_t = N_t/N_c$, $K_n = N_n/N_c$ y $K_s = N_s/N_c$ son básicamente constantes entre los distintos conjuntos de datos con los que vamos a trabajar.

Por lo tanto vamos a trabajar con un único tamaño de la entrada, n , el **número de cruces**. Para normalizar las medidas de tiempo entre distintos conjuntos de datos podemos aplicar factores de corrección según nuestra estimación de la fórmula.

	$N_c \equiv n$	N_t	N_n	N_s	λ_t	λ_n	λ_s
Zona1	790	1.810	12	6	1,0375	1,1378	1,3049
Zona2	2.049	4.811	38	23	1,0124	0,9319	0,8829
Zona3	4.805	11.319	89	58	1,0091	0,9331	0,8210
Zona4	10.387	24.673	176	113	1,0007	1,0200	0,9110
Zona5	20.483	48.689	354	203	1,0000	1,0000	1,0000
Zona6	31.350	73.839	631	166	1,0092	0,8587	1,8717
Zona7	133.652	313.137	1.274	629	1,0146	1,8131	2,1058
Zona8	578.114	1.289.145	6.384	3.909	1,0660	1,5651	1,4657

Tab. 3 Factores de corrección de tiempos para cada conjunto de datos

En la Tabla 3 se muestran los tamaños de los distintos conjuntos de datos y los factores de corrección, donde la base es la medida de la Zona 5. Si, por ejemplo, estimamos que una medida puede ser de orden $O(N_c \cdot N_t \cdot N_s^2)$ entonces multiplicaríamos el tiempo medido por los factores $\lambda_t \cdot \lambda_s^2$ correspondientes a cada conjunto de datos.

2.10. Comprobación de la solución

En la clase **SolucionAbs** se proporciona el método **EscribeResultados** que permite generar dos ficheros (`res_ser.txt` y `res_nuc.txt`) que podéis comparar con las soluciones reales (ficheros `sol_ser.txt` y `sol_nuc.txt`).

3. Objetivos

La aplicación pedirá al usuario el directorio donde se encuentran los datos (que os podéis descargar del Campus Virtual), el número de repeticiones y si se escriben o no los resultados a fichero (ver apartado 2.10) y escribirá por pantalla los tiempos de cada una de las etapas (lectura de ficheros, cálculo de las listas de adyacencia, resolución del primer problema y resolución del segundo problema).

Se deberán obtener datos para los conjuntos de datos proporcionados (Zonas 1 a 7). Es posible que para la **Zona 7** y el segundo problema la aplicación tarde tanto tiempo que solo pueda proporcionarse una estimación (medir el tiempo para unos pocos servicios y extrapolar para los restantes). Estos datos se analizarán para generar un informe en el que se incluya:

- La complejidad temporal de cada etapa, en función de n (número de cruces), con el término de mayor crecimiento expresado de manera exacta y el resto en notación asintótica.
- Una propuesta, basada en los resultados experimentales y la inspección del código, del orden de cada etapa en función de los cuatro tamaños de la entrada, en notación asintótica (por ejemplo, $O(N_c \cdot N_n^{3/2} + N_t^2 \cdot N_s)$).
- La extrapolación del tiempo que tardaría cada etapa con la **Zona 8**.
- Las tablas y gráficas que se han usado para obtener los resultados.

El informe se presentará en un documento PDF, junto con el código, mediante una tarea que se abrirá en el Campus Virtual.

4. Defensa y evaluación de la Práctica

En la defensa se pedirá: O bien una modificación sencilla de vuestro código o bien una recogida y análisis rápido para obtener una estimación de la eficiencia de un algoritmo (que puede o no estar relacionado con esta práctica).

La evaluación de la práctica se divide en dos etapas:

1. Presentación electrónica del documento PDF con el análisis y de los ficheros ***.java** que contienen el código del programa creado para obtener las medidas. Se habilitará en el Campus Virtual de la asignatura una tarea de subida de ficheros cuya fecha límite será el **domingo 8 de noviembre a las 23:59**. Al principio de todos los documentos y ficheros debe aparecer un comentario con el nombre de quienes la han desarrollado.
2. Evaluación **presencial**, en laboratorio, ante el profesor. Se realizará en el lugar, día y hora correspondiente al horario de prácticas del subgrupo al que pertenezca durante la semana del 9 al 13 de noviembre.

Es en la defensa de la práctica donde se produce la evaluación, la presentación electrónica es simplemente un requisito previo para garantizar la equidad entre subgrupos y la comprobación preliminar de autoría.

Nota: El no poder realizar la modificación y análisis requerido puede dar lugar (según las circunstancias) a que se considere la práctica como no presentada por no haber podido acreditar su autoría.